

Python Programs Documentation & Guide

Bank System | Calculator | Number Guessing Game

Complete Code Review with Explanations
Ready to Push to GitHub

Table of Contents

1. Bank System Using Classes
2. Calculator Using Class
3. Number Guessing Game
4. How to Push to GitHub

1. Bank System Using Classes

Overview:

This program implements a simple bank account management system using Object-Oriented Programming (OOP). It allows users to deposit money, withdraw money, and check their account balance through an interactive menu.

Code:

```
class Bank:
    def __init__(self):
        self.balance = 0

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            print(f"Withdrawn {amount}, Balance: {self.balance}")
        else:
            print("Insufficient funds")

    def display(self):
        print("Balance:", self.balance)

# Menu
bank = Bank()
while True:
    choice = input("\n1.Deposit 2.Withdraw 3.Display 4.Exit: ")
    if choice == '1':
        bank.deposit(float(input("Amount: ")))
    elif choice == '2':
        bank.withdraw(float(input("Amount: ")))
    elif choice == '3':
        bank.display()
    elif choice == '4':
        break
    else:
        print("Invalid choice")
```

Detailed Explanation:

Class Definition: The Bank class encapsulates all banking operations.

Constructor (__init__): Initializes balance to 0 when a Bank object is created.

deposit() method: Adds the specified amount to the balance.

withdraw() method: Checks if sufficient funds exist before deducting amount. Shows error if funds are insufficient.

display() method: Prints the current balance.

Menu System: A while loop provides user interaction with choices 1-4 for different operations.

Key Concepts Used:

Classes, Methods, Conditional Statements, Loops, User Input

2. Calculator Using Class

Overview:

This program implements a calculator using a class-based approach. It performs basic arithmetic operations (addition, subtraction, multiplication, division) and uses Python's `hasattr()` and `getattr()` methods for dynamic method selection.

Code:

```
class calculation:

    def add(self , num1 , num2):
        return num1+num2

    def sub(self , num1 , num2):
        return num1-num2

    def mul(self , num1 , num2):
        return num1 * num2

    def div(self , num1 , num2):
        if num1==0:
            print("The number can be divided because the num1 value is 0")
            return none
        else :
            return num1 / num2

calc = calculation()

num1 = int(input("Enter a number "))
num2 = int(input("Enter a number "))
choice = input(" Select choice of add \n sub\n mul\n div\n " )
if hasattr(calc, choice):
    method = getattr(calc, choice)
    result = method(num1, num2)
    if result is not None:
        print(f"Result: {result}")
else:
    print("Invalid choice! Please select add, sub, mul, or div.")
```

Detailed Explanation:

Class Methods: Four methods (add, sub, mul, div) perform different mathematical operations.

Division Safety Check: The `div()` method checks if `num1` is 0 before division (though logically it should check `num2`).

hasattr(): Checks if the `calculation` object has an attribute/method with the name provided by the user.

getattr(): Retrieves the method from the object dynamically based on user choice.

Null Check: Verifies the result is not `None` before printing to handle invalid operations.

Key Concepts Used:

Classes, Methods, `hasattr()`, `getattr()`, Conditional Logic, Return Values

3. Number Guessing Game

Overview:

This program implements an interactive number guessing game where the player has 7 chances to guess a random number within a user-defined range. The game provides hints (higher/lower) to help the player.

Code:

```
import random

low = int(input("Enter a lower number "))
high = int(input("Enter a higher number "))

print(f" You need to guess the number from {low} to {high} in 7 chances")

num = random.randint(low, high)
ch = 7
left = 0

while left < ch :
    left +=1
    guess=int(input("Guess the number : "))

    if( guess==num):
        print(f"Your guess is right. Completed in {left} chances. Number: {num}.")
        break

    elif (left<ch & guess!=num):
        print(f"Your Guess is wrong, You have {7-left} chances left")

    elif (guess>num):
        print(f"Your Guess is high, Try lower Number than {high}")

    elif (guess<num):
        print(f"Your Guess is low, Try higher Number than {low}")
```

Detailed Explanation:

Random Module: Generates a random integer between low and high using random.randint().

Loop Control: A while loop runs for maximum 7 iterations (attempts).

Comparison Logic: Compares user's guess with the random number to provide feedback.

Hint System: Tells the player if their guess is too high or too low.

Win Condition: When guess matches the number, displays success message and breaks the loop.

Key Concepts Used:

Random Module, Loops, Conditional Statements, User Input, String Formatting

4. How to Push to GitHub

Step 1: Create a GitHub Account and Repository

1. Go to www.github.com and create an account (if you don't have one) 2. Click the '+' icon in the top-right corner → Select 'New repository' 3. Name your repository (e.g., 'python-programs' or 'learning-python') 4. Add a description (optional) 5. Choose 'Public' to make it visible to everyone 6. Click 'Create repository'

Step 2: Install Git

On Windows: Download from www.git-scm.com and run the installer **On Mac:** Install via Homebrew: `brew install git` **On Linux:** `sudo apt-get install git`

Step 3: Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Step 4: Initialize Repository Locally

```
# Navigate to your project folder
cd path/to/your/project

# Initialize git repository
git init

# Add all Python files
git add *.py

# Or add specific files
git add bank_system_using_classes.py
git add calculator_using_class.py
git add Number_Guessing.py
```

Step 5: Create Initial Commit

```
git commit -m "Initial commit: Add Python programs with documentation"
```

Step 6: Connect to Remote Repository

Copy the repository URL from GitHub and run:

```
git remote add origin https://github.com/YOUR_USERNAME/your-repo.git
```

Step 7: Push to GitHub

```
git branch -M main
git push -u origin main
```

Recommended: Create a README.md File

Create a file named README.md in your project folder with:

```
# Python Programs Collection

This repository contains three Python programs demonstrating OOP concepts.

## Programs Included

1. **Bank System** - Simple bank account management
2. **Calculator** - Basic arithmetic operations
3. **Number Guessing Game** - Interactive guessing game

## Usage

Run any program with:
```
python program_name.py
```

## Requirements
- Python 3.6 or higher

## Author
[Your Name]
```